Health Endpoints

live, ready, health: three different questions

Watch the 4-min video

In one line: `/live` says the process isn't stuck, `/ready` says this instance can serve right now, and `/health` is a detailed report; keep readiness about local essentials, cheap and cached, so one shared-dependency blip can't pull every instance out of service.

1 Trivial endpoints, real outages

A new pod starts and the load balancer sends it traffic at once, but its database pool isn't connected yet: the first hundred requests fail. Or a health check that queries the database every second, from every pod, becomes that database's biggest load. Health endpoints look trivial; the details decide whether they prevent outages or cause them. (Lesson 9 covered health checks and self-healing; this lesson designs the endpoints.)

2 Three endpoints, three questions

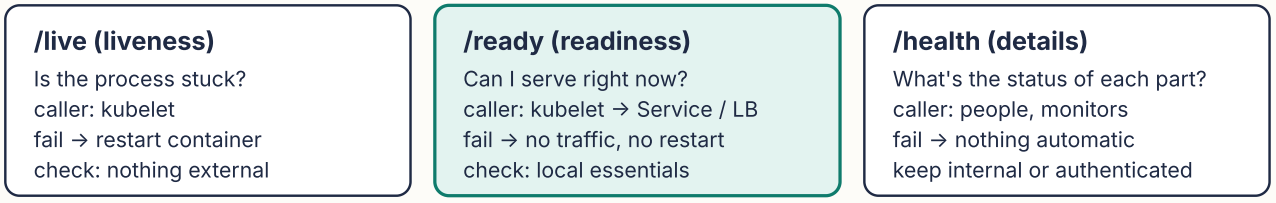


Figure 1. Different callers poll each endpoint and take different actions. Mixing them up (e.g. checking the DB in liveness) causes restart storms.

In Kubernetes these map to **liveness**, **readiness** and **startup** probes. Defaults: `periodSeconds: 10`, `timeoutSeconds: 1`, `failureThreshold: 3`. A failed liveness probe **restarts** the container; a failed readiness probe only removes the pod from Service endpoints. A **startup probe** holds off the other two until slow initialisation (e.g. loading a model) finishes, so liveness doesn't kill a pod that is merely starting.

3 What readiness should check

What *this instance* needs to do its job: DB pool connected, caches warmed, config loaded, model in memory. Return **200** when ready, **503** when not. On shutdown (`SIGTERM`), flip to not-ready first, keep serving in-flight requests while the load balancer stops sending new ones, then exit before the grace period (`terminationGracePeriodSeconds`, default 30 s) runs out.

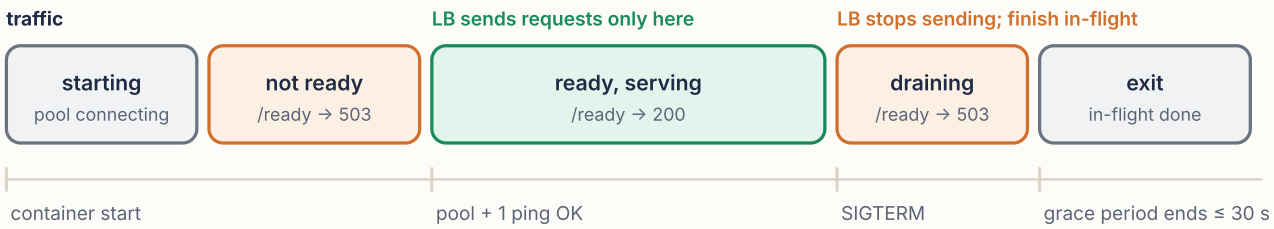


Figure 2. Readiness gates traffic at both ends of a pod's life: no requests before it's warm, none after it starts draining.

4 Shared dependencies: don't amplify outages

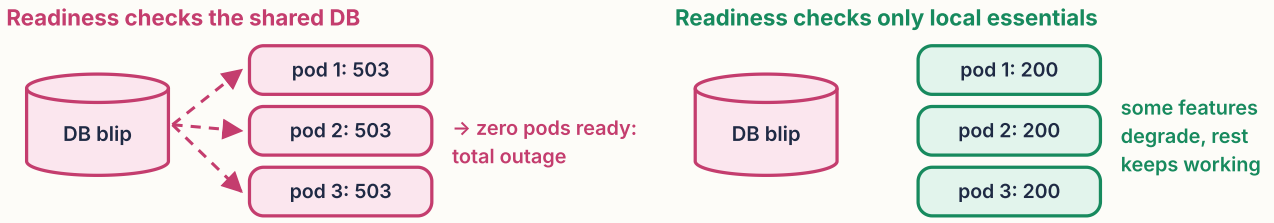


Figure 3. If readiness fails whenever a shared dependency fails, every pod goes unready at once and a partial outage becomes a total one.

Check only what is critical to *this* instance and that a restart or reroute could fix. If the database is down for everyone, removing all pods helps nobody: better to stay ready, return fast errors or fallbacks for affected features (lesson 8), and let symptom alerts (lesson 67) fire.

5 Cheap, cached and private

- **Cheap:** no real query per probe. Use a pool ping or reuse the result of recent real traffic.
- **Cached:** refresh dependency status in the background every few seconds; the endpoint returns the cached answer.
- **Short timeouts** inside the check, so a slow dependency can't hang the endpoint past the probe's own 1 s timeout.
- **Private details:** versions, hostnames and failing dependencies help attackers. Public answer = status code only; details on an internal port or behind auth.

	/live	/ready	/health
Checks	event loop responsive	pool, warm cache, config	each dependency, version
Response	200 / failure	200 or 503	JSON per component
Exposure	cluster-internal	cluster-internal	internal port or authenticated

WORKED EXAMPLE: FASTAPI THAT NEVER SERVES COLD

A lifespan/startup hook creates the DB pool, runs one `SELECT 1`, then sets `READY`. `@app.get("/ready")` returns `Response(status_code=200 if READY.is_set() else 503)`; `/live` just returns 200. Kubernetes points `readinessProbe` at `/ready` and `livenessProbe` at `/live`, so no request ever hits a cold pod. On SIGTERM the app clears `READY` and drains. A detailed `/health`, with per-dependency status, is served on an internal admin port.

COMMON MISTAKES

- Checking the database in **liveness**: a DB blip restarts every pod, which then all reconnect at once.
- One `/health` used for everything, so liveness, readiness and humans all get the same (wrong) answer.
- No readiness gate, so pods receive traffic before pools and caches are ready.
- Detailed health JSON on the public internet.

6 Key terms

Liveness probe: is the process alive and not stuck; failure restarts it

Startup probe: delays other probes during slow startup

503: Service Unavailable: the not-ready answer

Readiness probe: can it serve now; failure removes it from rotation

Draining: finishing in-flight work after leaving rotation

Grace period: time between SIGTERM and SIGKILL (default 30 s)

7 Interview questions

Q1. What should a readiness endpoint check? BASIC

Whether this instance can serve right now: local essentials like its DB pool, warm caches and loaded config, using cheap, cached checks with short timeouts; not every blip of a shared dependency.

Q2. Why not expose detailed health publicly? BASIC

It leaks versions, hostnames and internal topology. Keep the public answer to a status code; put details behind authentication or on an internal port.

Q3. Pods restart in a loop whenever the database is slow. Why? SCENARIO

The liveness probe checks the database, so DB slowness looks like a dead process. Make liveness local only; use readiness or fallbacks for dependencies.

Q4. Users see errors during every deploy. What do you check? SCENARIO

That readiness gates traffic until warm-up completes, and that on SIGTERM the app fails readiness and drains in-flight requests before exiting.

Q5. Should readiness fail when a shared database is down? TRICKY

Usually not: all pods would go unready together, turning partial impact into a total outage. Stay ready and degrade the affected features.

Q6. Why would you need a startup probe if you have liveness? TRICKY

Slow starters (big models, cache warm-up) would fail liveness and be killed before they finish; the startup probe disables liveness until startup succeeds.

Next: SD-N-011 · Root-Cause Analysis backendroadmap.com